

Brandon Nguyen, 827813045
COMPE596 - Spring 2026
2/23/2026

COMPE596 P04 - Nvidia CUDA Reduction By Summation

1) Code Listing

// Main Program

```
#include <cuda.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define BLOCK_SIZE 512

__global__ void reduction(float *input, float *output, int len)
{
    __shared__ float partialSum[2 * BLOCK_SIZE];
    unsigned int t = threadIdx.x;
    unsigned int start = 2 * blockIdx.x * BLOCK_SIZE;

    if (start + t < len)
        partialSum[t] = input[start + t];
    else
        partialSum[t] = 0;

    if (start + BLOCK_SIZE + t < len)
        partialSum[BLOCK_SIZE + t] = input[start + BLOCK_SIZE + t];
    else
        partialSum[BLOCK_SIZE + t] = 0;

    for (unsigned int stride = BLOCK_SIZE; stride > 0; stride >>= 1)
    {
        __syncthreads();
        if (t < stride)
            partialSum[t] += partialSum[t + stride];
    }

    if (t == 0)
```

```
    output[blockIdx.x] = partialSum[0];  
}
```

```
int main(int argc, char *argv[])  
{  
    if (argc != 2)  
    {  
        printf("Usage: %s N\n", argv[0]);  
        return 0;  
    }
```

```
    int N = atoi(argv[1]);  
    size_t bytes = N * sizeof(float);
```

```
    float *h_input = (float *)malloc(bytes);
```

```
    for (int i = 0; i < N; i++)  
        h_input[i] = (float)(i + 1);
```

```
    clock_t t;  
    double cpu_sum = 0.0;
```

```
    t = clock();  
    for (int i = 0; i < N; i++)  
        cpu_sum += h_input[i];  
    t = clock() - t;
```

```
    double cpu_time = ((double)t / CLOCKS_PER_SEC) * 1000.0;  
    printf("CPU Sum: %.0f\n", cpu_sum);  
    printf("CPU Time: %f ms\n", cpu_time);
```

```
    float *d_input;  
    cudaMalloc((void **)&d_input, bytes);
```

```
    cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice);
```

```

int currentSize = N;
float *d_in = d_input;
float *d_out;

cudaEvent_t start, stop;
float gpu_time;

cudaEventCreate(&start);
cudaEventCreate(&stop);
cudaEventRecord(start, 0);

while (currentSize > 1)
{
    int numBlocks = (currentSize + (2 * BLOCK_SIZE - 1)) / (2 * BLOCK_SIZE);
    size_t outBytes = numBlocks * sizeof(float);

    cudaMalloc((void **)&d_out, outBytes);

    reduction<<<numBlocks, BLOCK_SIZE>>>(d_in, d_out, currentSize);
    cudaDeviceSynchronize();

    if (d_in != d_input)
        cudaFree(d_in);

    d_in = d_out;
    currentSize = numBlocks;
}

cudaEventRecord(stop, 0);
cudaEventSynchronize(stop);
cudaEventElapsedTime(&gpu_time, start, stop);

float gpu_sum;
cudaMemcpy(&gpu_sum, d_in, sizeof(float), cudaMemcpyDeviceToHost);

printf("GPU Sum: %.0f\n", gpu_sum);
printf("GPU Time: %f ms\n", gpu_time);

printf("Speedup (CPU/GPU): %f\n", cpu_time / gpu_time);

```

```
    cudaFree(d_in);
    cudaFree(d_input);
    free(h_input);

    return 0;
}
```

// Makefile

```
NVCC = nvcc
TARGET = P04
SRC = P04.cu
```

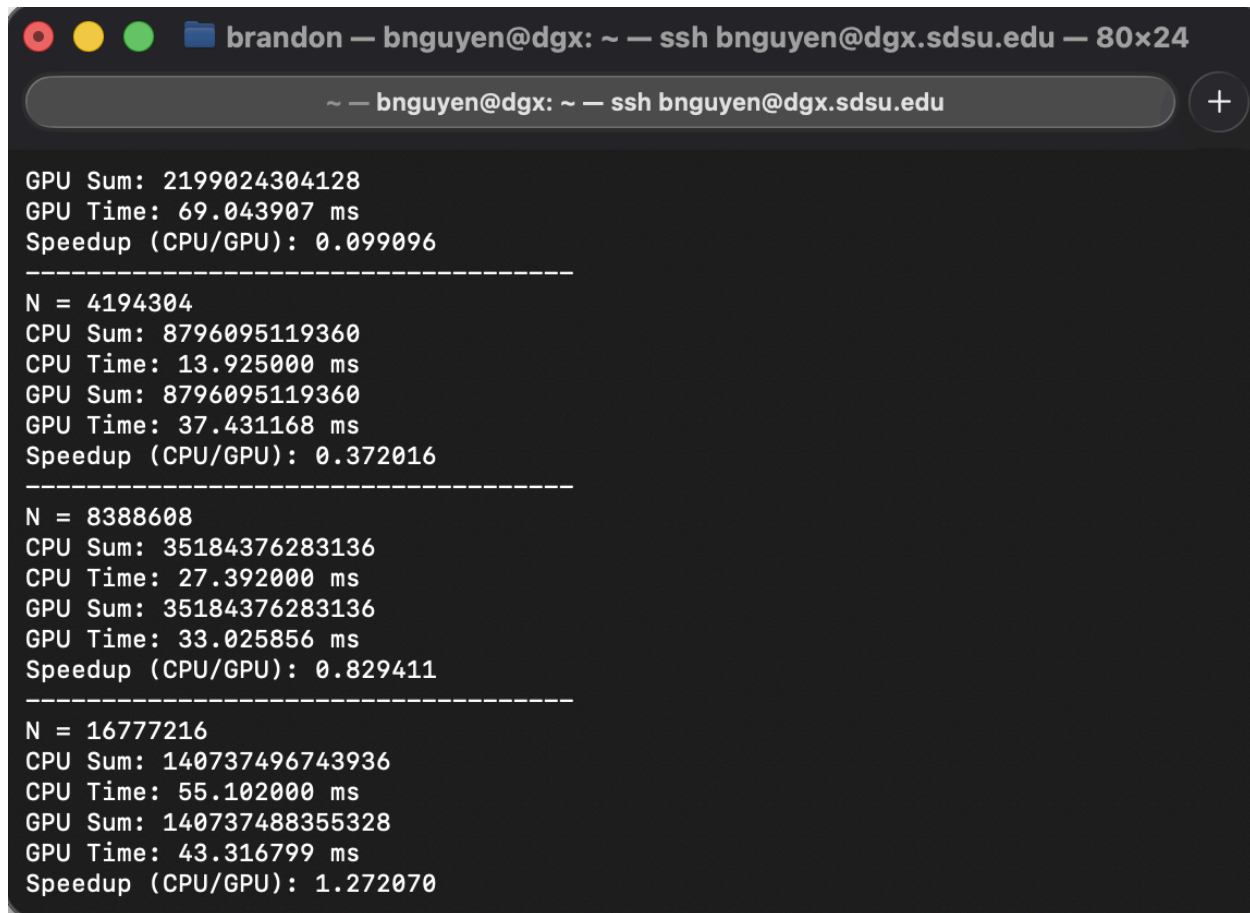
```
all: $(TARGET)
```

```
$(TARGET): $(SRC)
    $(NVCC) $(SRC) -o $(TARGET)
```

```
run: $(TARGET)
    @echo "Running benchmark tests..."
    @for i in `seq 10 24`; do \
        N=`echo "2^$$i" | bc`; \
        echo "-----"; \
        echo "N = $$N"; \
        ./$$(TARGET) $$N; \
    done
```

```
clean:
    rm -f $(TARGET)
```

2) Program Output



```
brandon — bnguyen@dgx: ~ — ssh bnguyen@dgx.sdsu.edu — 80x24
~ — bnguyen@dgx: ~ — ssh bnguyen@dgx.sdsu.edu

GPU Sum: 2199024304128
GPU Time: 69.043907 ms
Speedup (CPU/GPU): 0.099096
-----
N = 4194304
CPU Sum: 8796095119360
CPU Time: 13.925000 ms
GPU Sum: 8796095119360
GPU Time: 37.431168 ms
Speedup (CPU/GPU): 0.372016
-----
N = 8388608
CPU Sum: 35184376283136
CPU Time: 27.392000 ms
GPU Sum: 35184376283136
GPU Time: 33.025856 ms
Speedup (CPU/GPU): 0.829411
-----
N = 16777216
CPU Sum: 140737496743936
CPU Time: 55.102000 ms
GPU Sum: 140737488355328
GPU Time: 43.316799 ms
Speedup (CPU/GPU): 1.272070
```

3) Discussion

Question: How long must an array be (as a power of 2) to see a runtime improvement of GPU device accelerated reduction over CPU sequential summation? (i.e., for which value of N (a power of 2) will elapsed time for host (CPU) summation exceed elapsed time for device (GPU) reduction summation?)

In P04, we are tasked with writing a program using the DGX server to observe CUDA reduction where the CUDA kernel reduces (by summation) an input array. Provided above is the main program, the makefile, and a picture of the terminal output. Most importantly, we are expected to compare the elapsed runtime between the CPU and GPU based on varying values of the array by powers of 2 to find the crossover and also write down the speedup as a ratio of time sequential over time parallel. To answer the question above, according to my terminal output screenshot, the crossover where the GPU becomes faster than the CPU is at the last value of 2^{24} from a tested range of 2^{10} to 2^{24} . This makes sense since we know that CPU is faster when the value of N is smaller, and GPU is supposed to be faster when the value of N increases, therefore,

the largest power of 2 in the tested range was when the crossover happened and GPU elapsed time was finally faster than the CPU. Overall, this fourth programming assignment was a good introduction to implementing CUDA programming code.